# Weekly Implementation Report

Implementation of an Open-Source SOC Lab
for Monitoring Attacks Against a Vulnerable REST API


| | |
|---|---|
| **Student:** | Bui Tat Dat |
| **Focus:** | Wazuh + TheHive + crAPI integration |
| **Report type:** | Weekly progress / implementation summary |
| **Date:** | April 17, 2026 |


**Purpose.** This report summarizes the current implementation status of the student's SOC laboratory, describes the deployed architecture, explains how logs flow from the target application to the monitoring platform, and identifies key technical gaps to be addressed in the next iteration.

## Contents

# 1 Executive Summary

The current implementation already forms a workable proof-of-concept SOC environment. The design combines three major building blocks: a single-node Wazuh stack for detection and centralized log analysis, a TheHive stack for alert and case management, and a segmented crAPI deployment that serves as the attack target. The architecture is strong because it does not place all services in one flat network. Instead, it separates Internet-facing, DMZ, backend, and database functions, which makes the lab more realistic and easier to explain from a defensive-security viewpoint.

At the present stage, the best-implemented detection path is the collection of Nginx logs from the public web component of crAPI through a dedicated Wazuh agent. This already allows the platform to observe behavior such as suspicious requests, repeated failures, brute-force attempts, abnormal API enumeration, or crafted payloads inside HTTP requests. However, the incident-response pipeline is still incomplete because direct integration from Wazuh to TheHive is not yet fully implemented. In addition, the current data collection depth is still limited because only the web-facing log path is clearly defined.

Overall, the implementation is suitable for a graduation-project prototype. The next improvements should focus on stabilizing inter-stack networking, increasing telemetry sources, and completing the SIEM-to-incident workflow.

# 2 Implementation Scope of This Week

This week's implementation covers the following practical tasks:

- Deployment of a single-node Wazuh platform including manager, indexer, and dashboard.
- Preparation of a TheHive environment using TheHive, Cassandra, Elasticsearch, and Nginx reverse proxy.
- Deployment of a segmented crAPI lab divided into router, DMZ, backend, and database layers.
- Configuration of a Wazuh agent to read shared Nginx log files produced by the public crAPI web service.
- Preparation for future correlation between Wazuh alerts and TheHive incident records.

# 3 System Architecture Overview

## 3.1 High-Level Architecture

Figure 1 presents the overall architecture of the current implementation. The design can be interpreted as three security-oriented layers:

- Target layer: the vulnerable application ecosystem represented by crAPI and its supporting services.
- Detection layer: Wazuh, which receives logs, applies analysis rules, and produces alerts.
- Incident-handling layer: TheHive, which is intended to receive escalated alerts and convert them into analyst-visible alerts or cases.

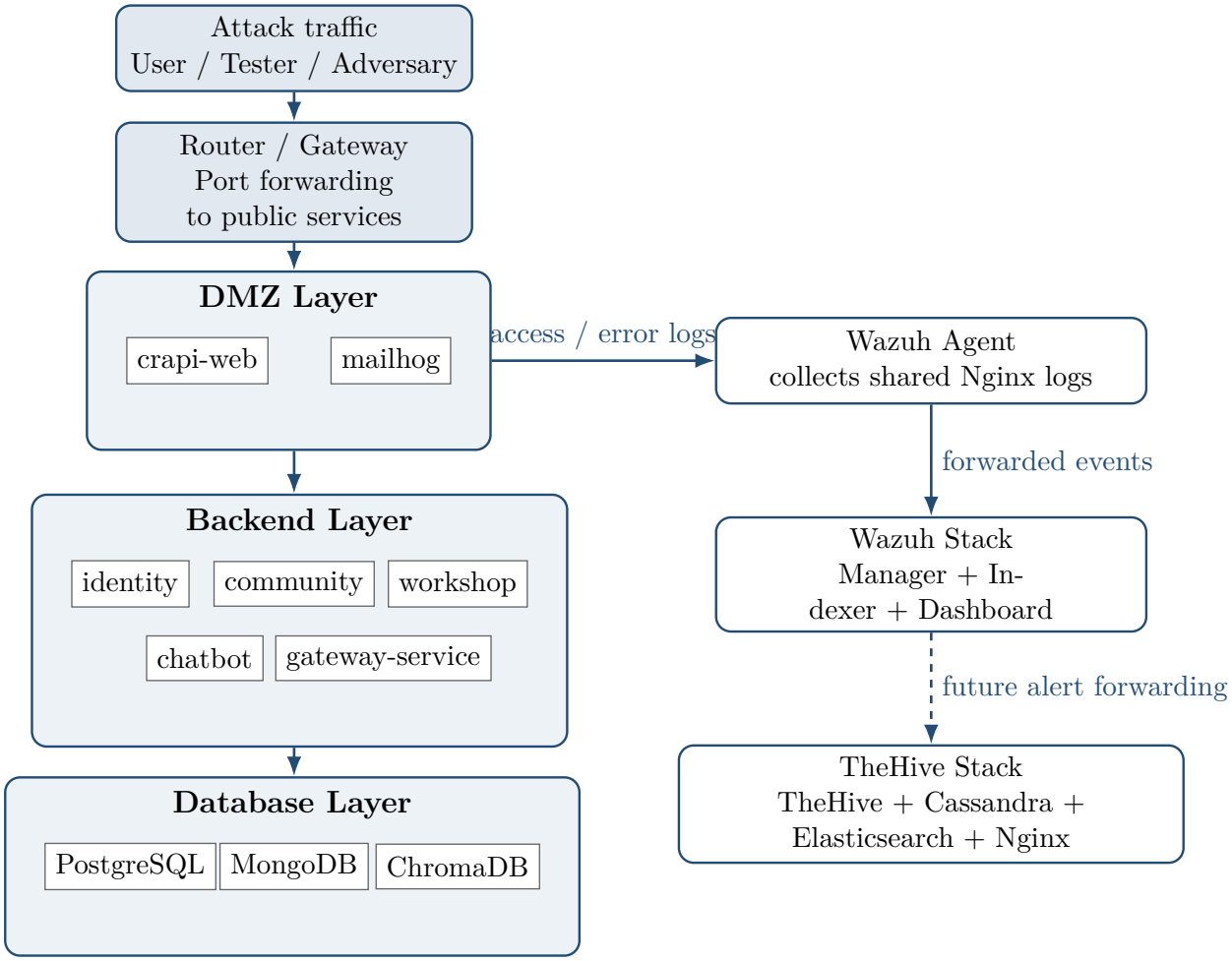


Figure 1: High-level architecture of the student's current SOC lab.

### 3.2 Network Segmentation Logic

The implementation uses multiple Docker networks to imitate layered network security. This is a good design choice because it supports the concept of defense in depth. Public traffic reaches the router first, then the public-facing web service in the DMZ. Internal application logic is located in backend networks, while persistence services remain in an internal database network. A dedicated SOC communication path is used so that monitored assets can communicate with Wazuh.

From an academic perspective, this segmentation is important because it allows the student to argue that the project is not only about collecting logs, but also about modeling a realistic enterprise-style application environment.

## 4 Detailed Component Architecture

### 4.1 Wazuh Stack

The Wazuh implementation contains three core services:

- Wazuh Manager: receives agent data, performs analysis, and exposes the Wazuh API.

- Wazuh Indexer: stores indexed security events for search and visualization.
- Wazuh Dashboard: provides the web interface for monitoring alerts and agent status.

This arrangement is appropriate for a student lab because it is lighter than a multi-node cluster and easier to operate on a single workstation. It also remains sufficiently realistic for demonstrating SIEM workflows.

### 4.2 TheHive Stack

TheHive is deployed with supporting services that reflect a small production-style setup:

- TheHive for alert and case management.
- Cassandra for backend data storage.
- Elasticsearch for indexing and search.
- Nginx for reverse proxy and front-end exposure.

This stack is especially valuable for the project because it provides a clear separation between detection and incident handling. Wazuh can be presented as the detection engine, while TheHive can be presented as the analyst-facing case-management system.

### 4.3 crAPI Laboratory Environment

The crAPI environment is the monitored target system. It includes a realistic set of dependent services rather than a single monolithic container. This is a strong point in the implementation because it produces richer traffic and allows the student to discuss attacks at multiple levels:

- web access and HTTP payload manipulation,
- authentication and identity abuse,
- backend service interaction,
- access to data-layer components.

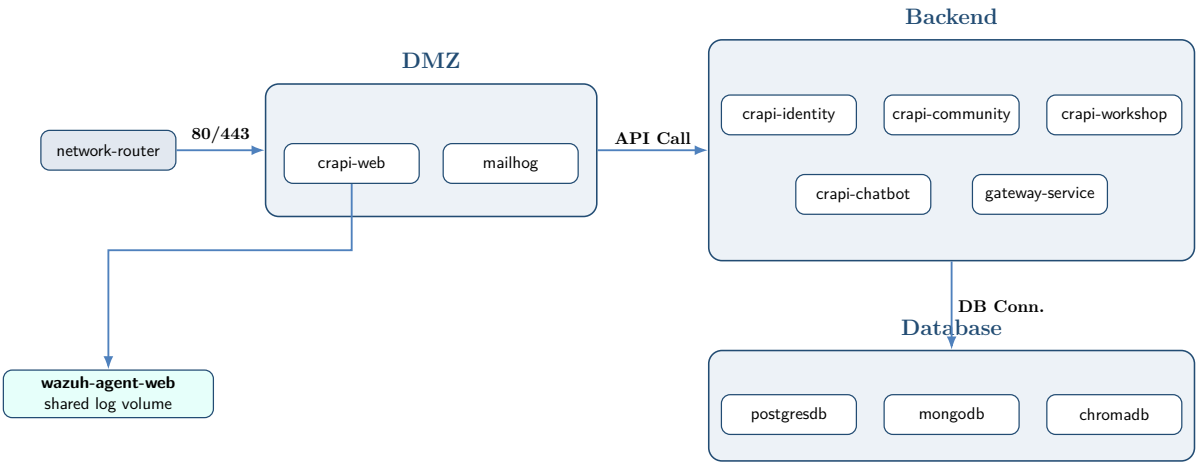


Figure 2: Final Standardized Architecture for crAPI Environment.

# 5 Data Flow and Monitoring Path

## 5.1 Current Operational Log Flow

The most important working data path in the current implementation is the monitoring of web logs. The public web component stores Nginx logs inside a shared volume, and the dedicated Wazuh agent mounts that same volume in read-only mode. This produces the following operational pipeline:

*Attack request → crAPI public web service → Nginx access/error logs → shared volume → Wazuh agent → Wazuh manager → Wazuh indexer/dashboard*

This path is already enough to demonstrate several forms of API-related suspicious activity:

- repeated authentication failures,
- abnormal request bursts from one IP,
- suspicious URI patterns,
- forced browsing or endpoint enumeration,
- malformed requests and server errors.

## 5.2 Target Future Response Flow

The intended end-to-end SOC workflow should extend the above path with an incident-management stage:

*Wazuh alert → filtering / severity logic → TheHive alert → analyst triage → case creation and response tracking*

This future path will strengthen the thesis because it turns the project from a pure monitoring system into a combined SIEM + incident response environment.

# 6 Implementation Assessment

## 6.1 Main Strengths

The implementation already has several strong design qualities:

1. Clear security layering. The target environment is segmented rather than flat.
2. Practical telemetry path. A real log source is already flowing into Wazuh.
3. Good tool separation. Wazuh handles detection while TheHive is prepared for case management.
4. Suitable scale for a thesis project. The architecture is realistic without being too heavy to operate.

### 6.2 Current Weaknesses and Risks

Several weaknesses should be documented honestly because they define the next engineering priorities:

1. Inter-stack networking is fragile. The current agent configuration appears to rely on a Compose-generated network name, which may change when the project directory or stack name changes.
2. Manager addressing is brittle. The agent points to a specific generated container name rather than a stable alias.
3. Telemetry depth is limited. The main monitored source is the Nginx log path of the web service; application logs, database logs, and host logs are not yet fully integrated.
4. Wazuh to TheHive forwarding is not yet complete. This means the case-management portion of the architecture remains partially conceptual.
5. Secrets are hardcoded in Compose files. This is acceptable for a temporary lab but should be acknowledged as a security limitation.

### 6.3 Component Summary Table

| Layer | Component(s) | Status | Role in the system |
|---|---|---|---|
| Target environment | crAPI web, identity, community, workshop, chatbot, gateway, supporting databases | Implemented | Generates vulnerable API traffic, authentication flows, backend communication, and storage interactions. |
| Detection | Wazuh manager, indexer, dashboard | Implemented | Centralized event ingestion, analysis, indexing, and visualization. |
| Log collection | Dedicated Wazuh web agent | Implemented | Reads shared Nginx logs from the crAPI web service and forwards them to Wazuh. |
| Incident management | TheHive, Cassandra, Elasticsearch, Nginx | Partially integrated | Prepared to manage alerts and cases, but forwarding from Wazuh is not yet complete. |
| Network design | DMZ, backend, database, SOC communication path | Implemented | Separates service roles and improves realism of the defensive architecture. |
| Automation / SOAR logic | Alert-to-case automation | Pending | Needed to transform selected Wazuh alerts into TheHive alerts or cases. |

## 7 Recommended Technical Improvements

### 7.1 Priority 1: Stabilize Inter-Stack Connectivity

A dedicated external Docker network with a fixed name should be created and shared across the Wazuh and crAPI stacks. This is better than relying on automatically generated network names. A stable alias such as `wazuh-manager` should also be assigned so that the agent configuration remains valid even if the deployment folder changes.

### 7.2 Priority 2: Add More Log Sources

To strengthen the thesis technically, the system should collect more than only web access logs. The most useful next sources are:

- application logs from identity or community services,
- container or Docker events,
- database logs where relevant,
- host-level audit or system logs.

This expansion is important because it enables better correlation between request behavior, business-logic outcomes, and system-level events.

### 7.3 Priority 3: Complete Wazuh-to-TheHive Integration

A custom integration script or webhook service should be added so that selected high-severity Wazuh alerts are sent to TheHive automatically. This step is essential if the final project title claims both monitoring and response support.

### 7.4 Priority 4: Develop API-Focused Custom Rules

The thesis will become much stronger if the student adds custom Wazuh rules specifically designed for API attacks. Good examples include:

- repeated 401 or 403 responses in short time windows,
- sequential access to many object identifiers from one source,
- suspicious access to sensitive endpoints,
- repeated request patterns consistent with brute force or endpoint discovery.

## 8 Planned Work for Next Week

The most logical plan for the next weekly cycle is:

1. create a fixed shared Docker network and clean manager aliasing,
2. verify the exact format of collected Nginx logs and tune the Wazuh agent configuration,
3. implement at least one custom rule set for suspicious API behavior,
4. create a webhook or integration script for forwarding important Wazuh alerts into TheHive,
5. demonstrate one complete test case from attack generation to incident creation.

## 9 Conclusion

This week's implementation establishes a solid technical foundation for the student's graduation project. The current architecture already demonstrates a meaningful combination of realistic target deployment, centralized monitoring, and planned incident handling. In particular, the segmented crAPI environment and the operational Wazuh ingestion path make the lab credible as a security-monitoring prototype.

The main challenge now is no longer basic deployment, but integration maturity. To reach a strong final thesis outcome, the system must move from a collection of working components to a coherent detection-and-response pipeline. Once networking is stabilized, telemetry is expanded, and TheHive integration is completed, the platform will be capable of supporting convincing demonstrations of API-centric attacks and SOC workflows.

## 10 Appendix: Student-Provided Implementation Basis

This report was written from the student's current Compose-based implementation, which includes a Wazuh single-node stack, a TheHive stack, and a segmented crAPI environment with a dedicated Wazuh agent reading shared Nginx logs. The appendix note is included to make clear that the report reflects the actual implementation already prepared by the student, not a purely hypothetical architecture.